# LLMs, Responsible AI & AI-Assisted Coding

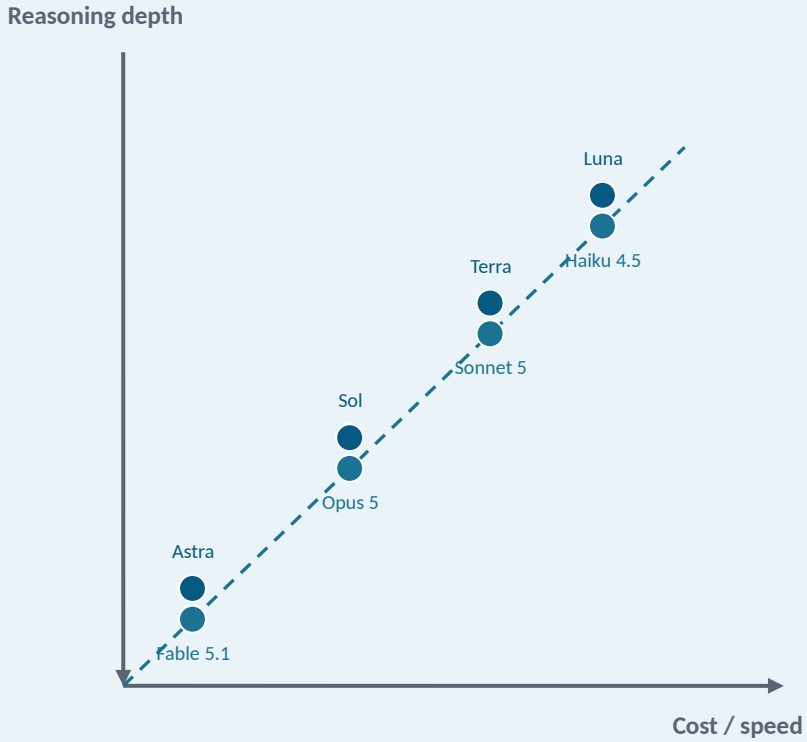
*Model Fit • Trustworthy Output • Safe Coding Practices*

MODEL FIT SAFETY CHECK CODE REVIEW

ORIGIN

# One Model Per Vendor Used to Be Enough

- Not long ago, choosing a vendor's model meant choosing one thing
- Today: a flagship reasoning tier, a primary tier, a cost-balanced tier, a cost-optimized tier
- The split exists because one model can't be both “good enough for the hardest case” and “cheap enough for every request”
- Splitting the catalog lets a team route each request to the model that matches its difficulty



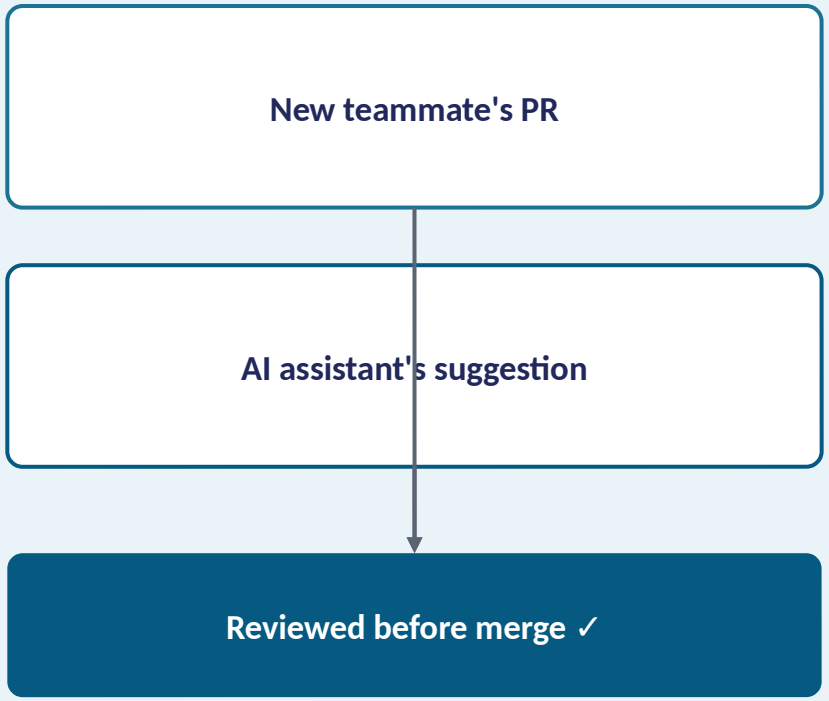

*Four tiers per vendor, spread along the same cost/reasoning trade-off*

*Sources: OpenAI model catalog; Claude model overview.*

# Review AI Code Like a New Contributor's Pull Request

ANALOGY

- A new teammate's first PR gets reviewed — not because it's assumed wrong
- Nobody has calibrated yet how much to trust that contributor's judgment
- An AI coding assistant's suggestion deserves the exact same posture
- GitHub's own guidance: “use it as a tool, not a replacement” for human judgment



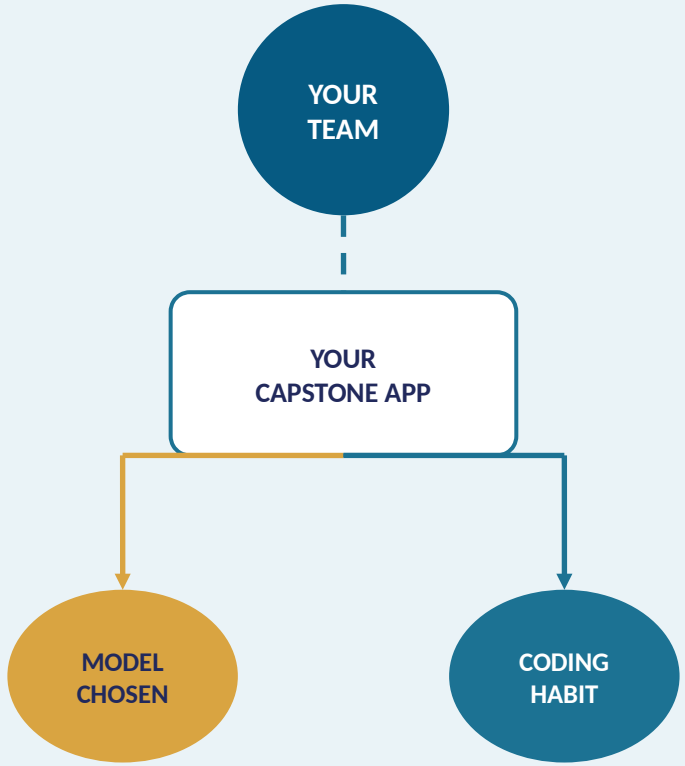

*Both routed through the same checkpoint before merge*

*Source: Responsible use of GitHub Copilot Chat in your IDE (docs.github.com).*

# Your Capstone Picks a Model and a Coding Habit

PROTAGONIST

- The same application and team from every earlier module, no new example
- Today it gains a deliberately chosen model behind its AI feature, not a default
- It also gains a documented, reviewed way of using an AI coding assistant
- Every problem from here on is something this app now needs, not a new scenario




*One team, one app — gaining a chosen model and a reviewed coding habit*

# LLM Capabilities, Limits & Responsible Use

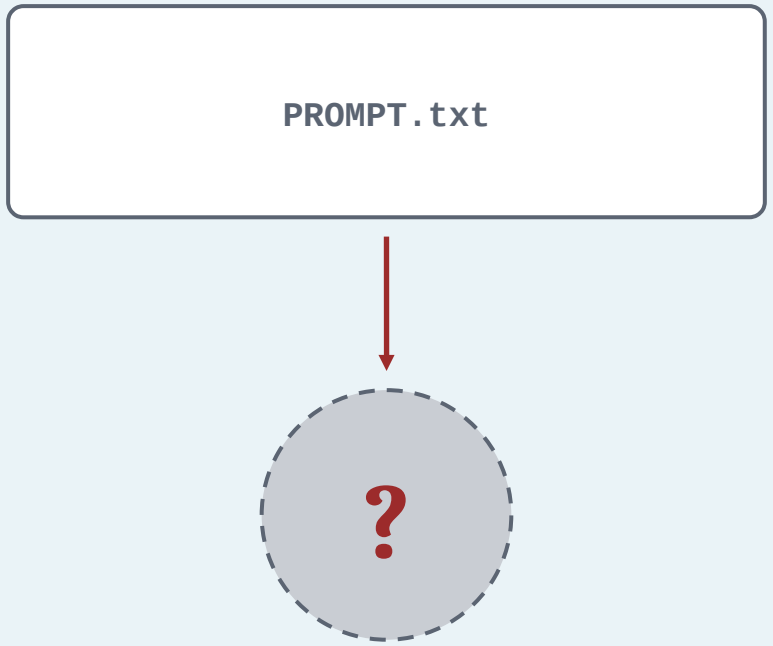
WHAT'S AHEAD

- Comparing OpenAI and Anthropic's current model families
- Hallucination risk and mitigation
- Responsible and ethical use of AI
- A safety and bias review checklist for AI-generated output

PROBLEM

# Which Model Actually Runs Your Prompt?

- Your feature's prompt is structured, guardrailed, and schema-constrained
- But it's been running against whichever model happened to be in the code sample
- That default is quietly deciding your feature's cost, speed, and reasoning quality
- Nobody on the team chose it on purpose

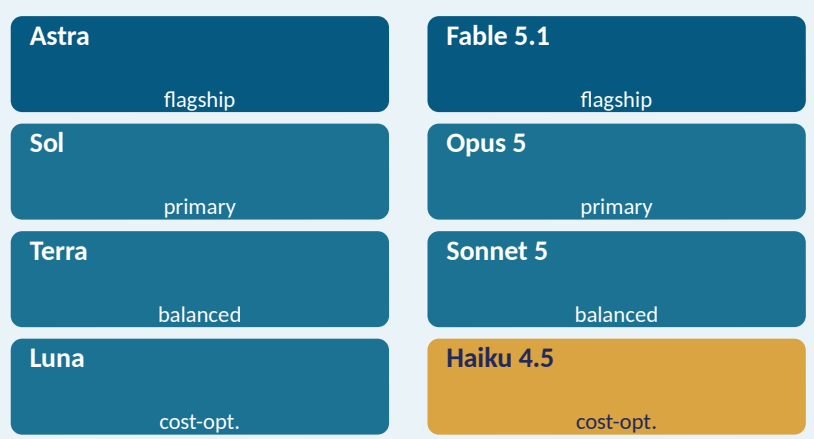



*The default nobody chose*

SOLUTION

# Comparing OpenAI & Anthropic's Model Families

- OpenAI: GPT-6 Astra (flagship) → GPT-5.6 Sol → GPT-5.6 Terra → GPT-5.6 Luna (cost-optimized)
- Anthropic: Claude Fable 5.1 (highest reasoning) → Opus 5 → Sonnet 5 → Haiku 4.5 (fastest)
- Most tiers share the same ~1M-token context window — the difference is reasoning depth and cost
- Haiku 4.5 is the one tier with a smaller, 200K-token context window too




*Four tiers per vendor — Haiku 4.5's rung marked for its smaller context window*

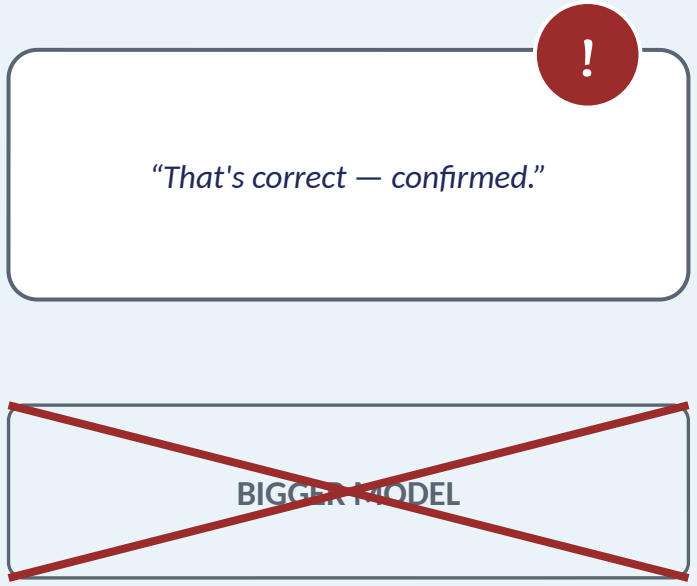
**HANDS-ON** Compare two models' reasoning, cost & hallucination trade-offs

*[Verified current: OpenAI and Claude model documentation, fetched this drafting pass.]*

PROBLEM

# Confident, Fluent, and Sometimes Wrong

- Every model above, even the most capable one, can state something false with full confidence
- A classifier that invents a policy detail that doesn't exist is a correctness bug
- Choosing a bigger, more capable model on its own doesn't fix this
- The team needs a real technique, not just “pick the smarter model”

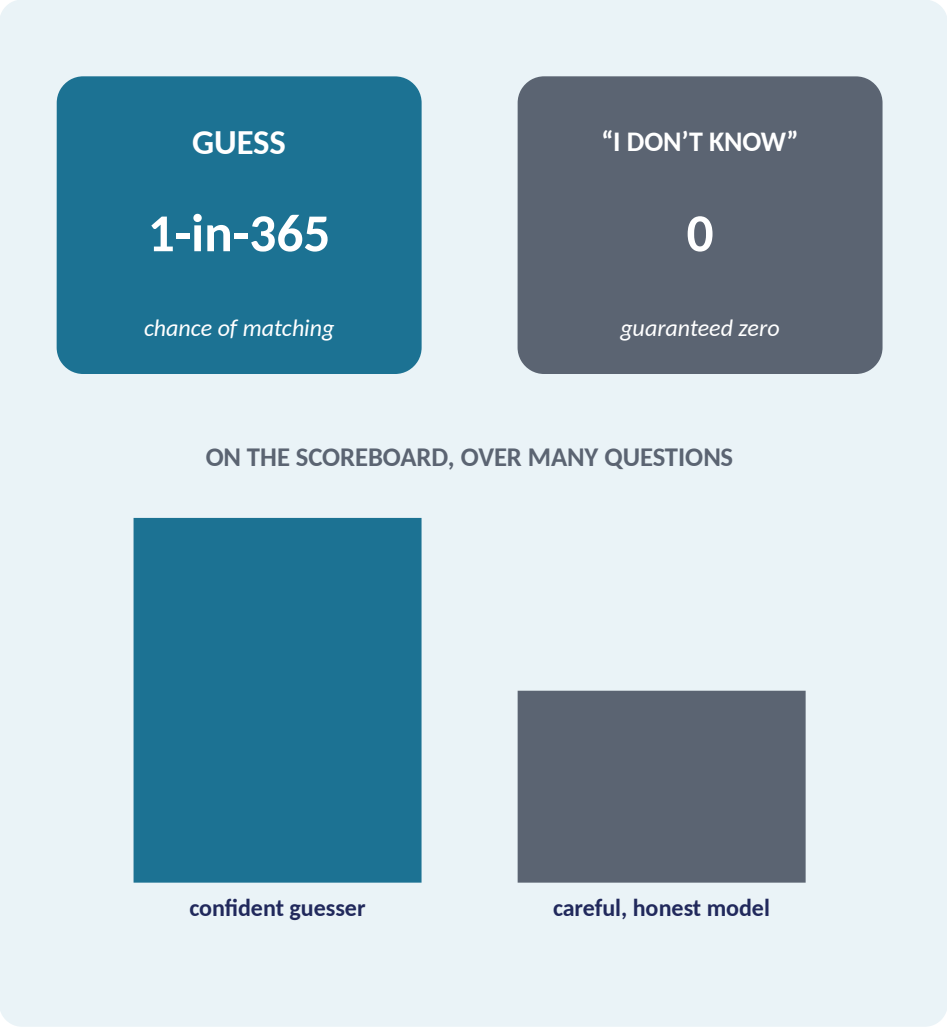



*A confident, fluent claim — with a hidden problem*

MECHANISM

# Why This Actually Keeps Happening

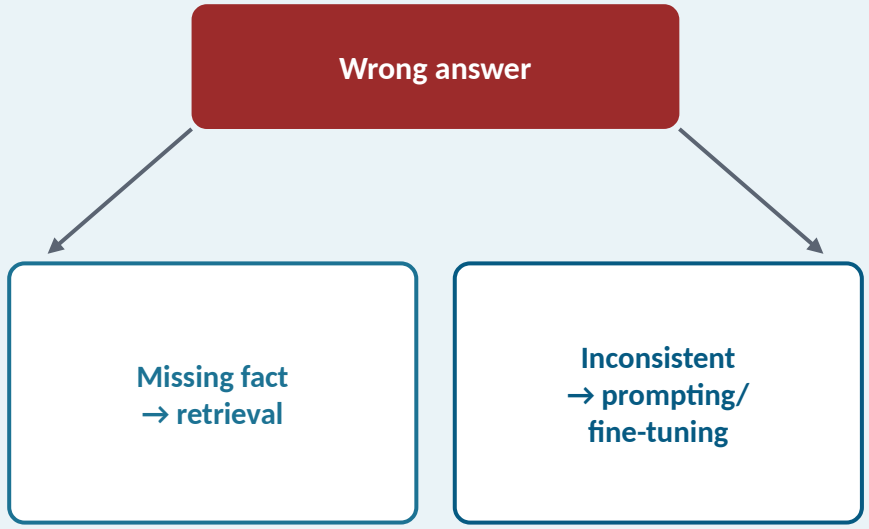
- Training grades models only on being exactly right — never on admitting “I don’t know”
- A specific guess has a real shot at scoring right; “I don’t know” always scores zero
- Across thousands of graded questions, confident guessers out-score careful, honest models
- An arbitrary fact can’t be reliably learned from text patterns alone — so the model still guesses fluently



SOLUTION

# Techniques to Reduce Hallucination

- Give the model explicit permission to say “I don't know”
- Ground it in direct quotes; require citations it must verify or retract
- Diagnose first: a missing fact (retrieval) vs. an inconsistent answer (prompting/fine-tuning)
- These reduce hallucinations — they don't eliminate them for high-stakes decisions



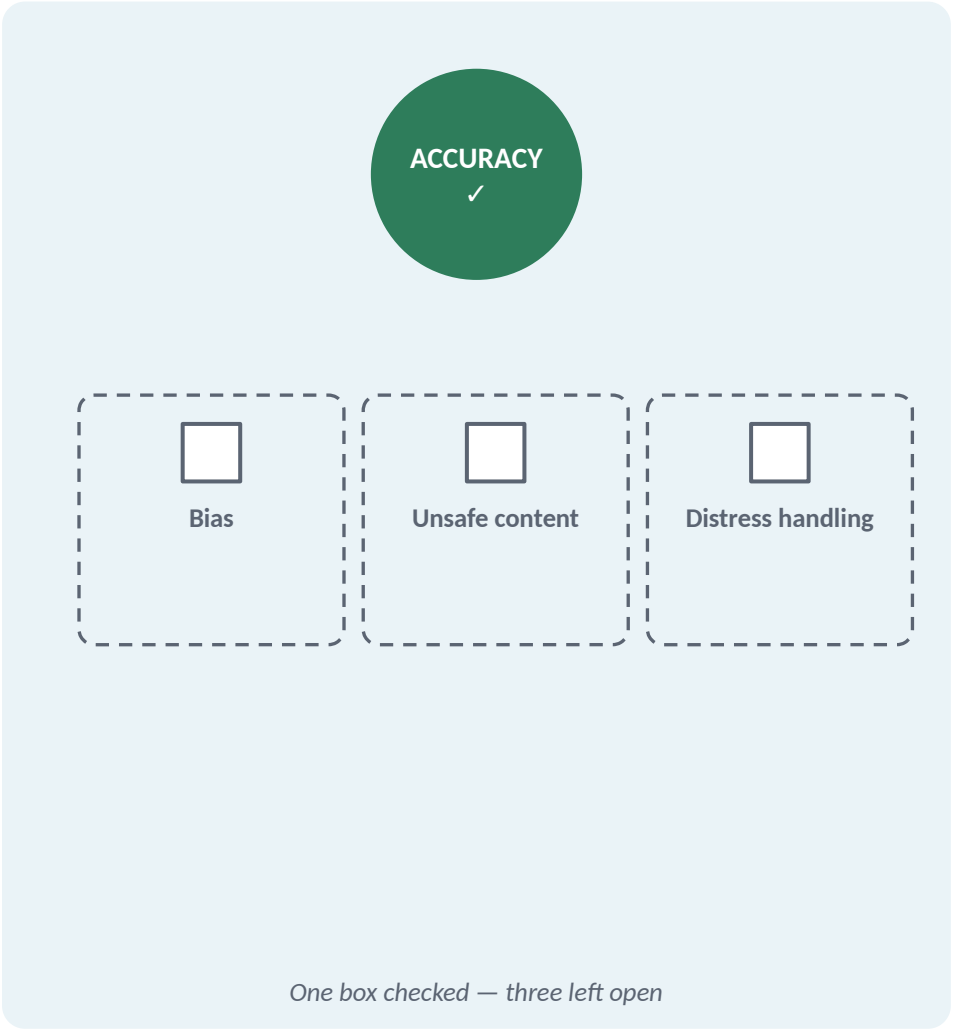

*Diagnose the failure type before picking a fix*

**HANDS-ON** Identify a hallucination risk specific to the capstone's use case

PROBLEM

# Beyond Getting Facts Wrong

- Hallucination is one specific failure mode — a wrong fact
- It's not the only way an AI feature can cause real harm
- Bias, unsafe content, and how the model handles a distressed user are separate risks
- A factually accurate classifier can still fail these standards




*One box checked — three left open*

SOLUTION

# Responsible & Ethical Use Standards

- Trained to be “politically evenhanded,” with a published bias-measurement methodology
- Apps serving minors: age verification, content moderation, child-privacy compliance
- A distressed message gets handled “with care,” pointed to real support, not treated as solved
- The real question: does your prompt and guardrail actually reflect these, in this feature's context


*(Slide 9 turns these standards into a repeatable practice)*

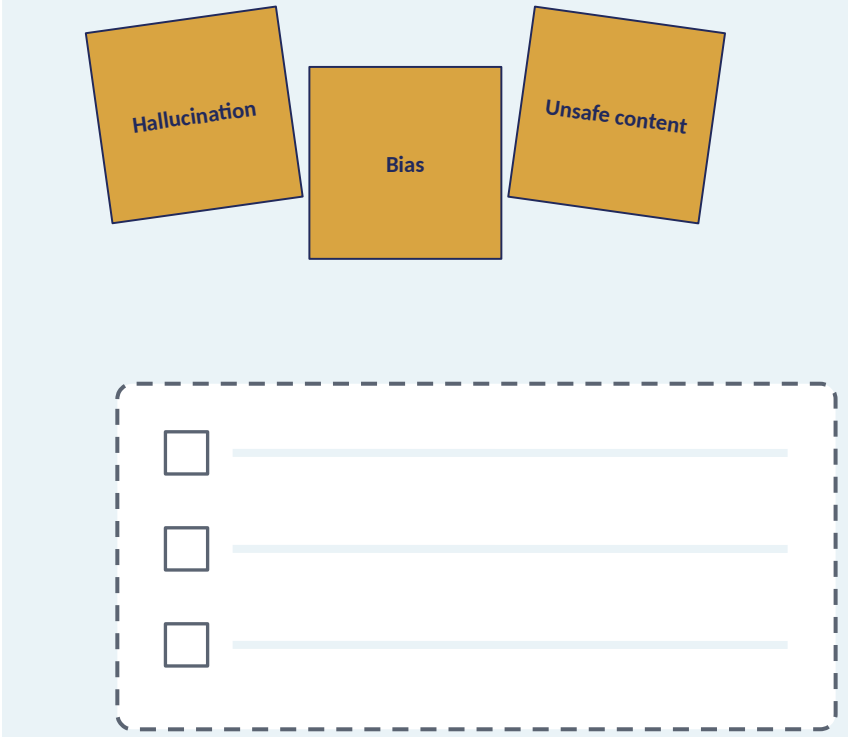
**HANDS-ON** Run the safety and bias checklist against an earlier model output

*Source: Anthropic usage policy (anthropic.com/policy).*

PROBLEM

# Knowing the Risks Isn't the Same as Checking for Them

- The last two topics named real risk categories: unreliable facts, and broader harms
- Naming a risk and actually checking an output against it are two different things
- Without a repeatable practice, the check only happens when someone happens to remember
- The team needs one checklist, run the same way every time

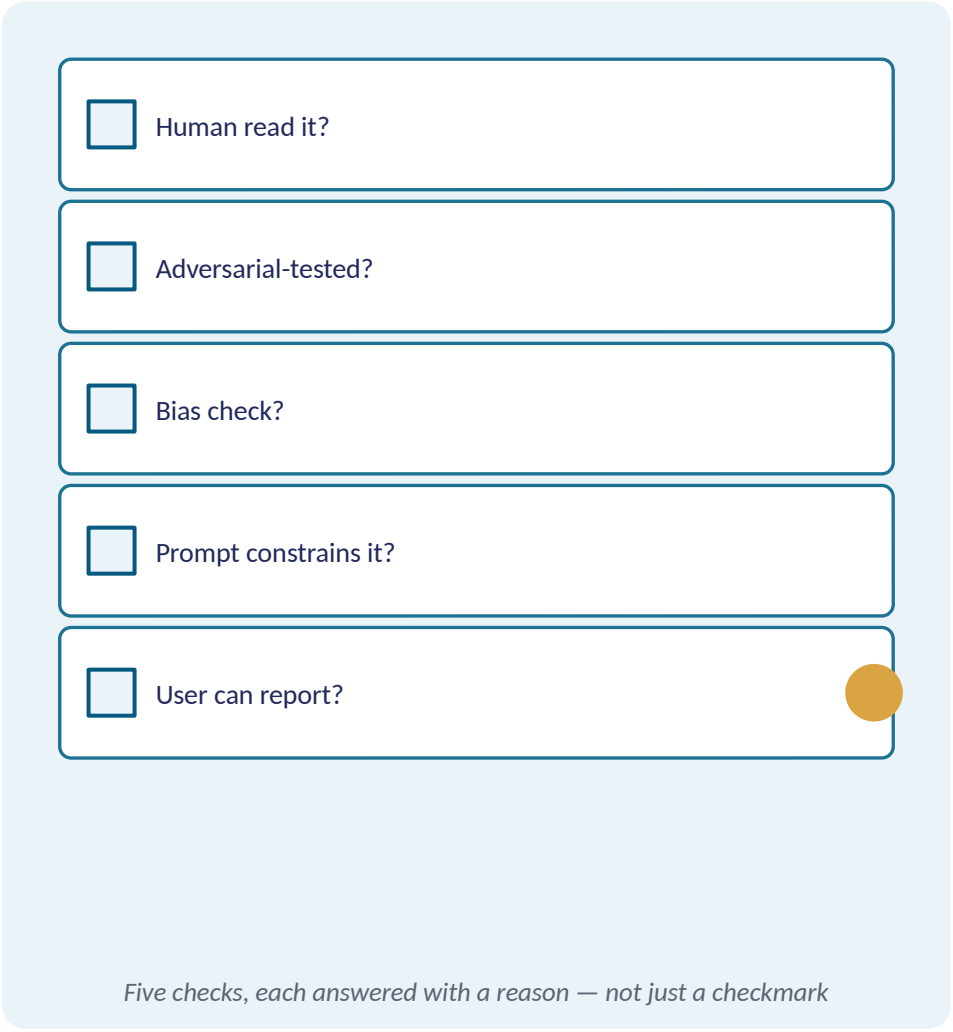



*Named risks, still no repeatable way to check them*

SOLUTION

# A Five-Question Safety & Bias Checklist

- Has a human actually read this, not just skimmed the confidence score?
- Was it tested against at least one adversarial or edge-case input?
- Does it show any sign of the bias the last topic named?
- Does the prompt already constrain tone/topic, or is safety doing the prompt's job?
- Is there a real, monitored way for a user to flag a bad output?




*Five checks, each answered with a reason — not just a checkmark*

*Sources: Claude reduce-hallucinations guide; OpenAI optimizing-accuracy & safety-best-practices guides; Anthropic usage policy.*

# AI-Assisted Coding, Testing & Data Privacy

WHAT'S AHEAD

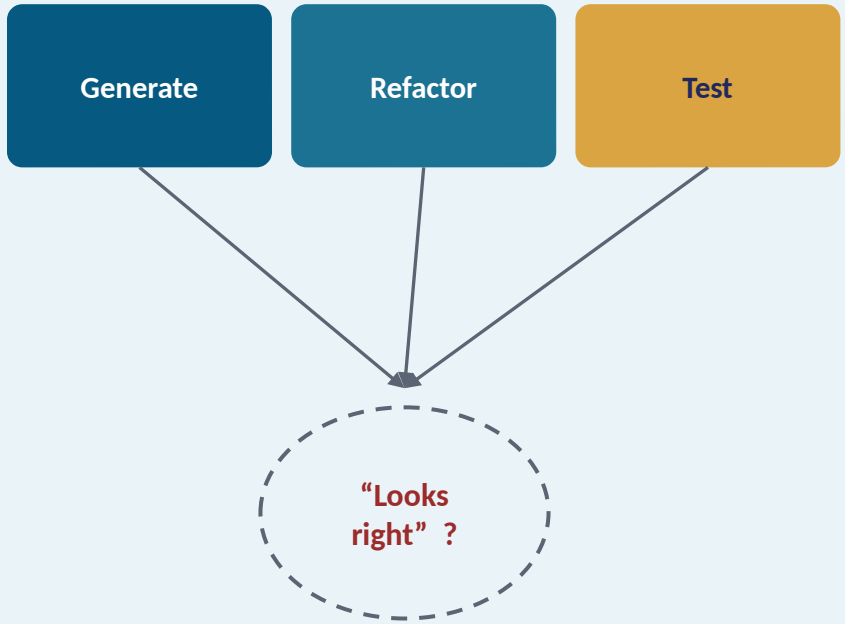
- AI-assisted coding workflows: generation, refactoring, test generation
- Output validation practices before merging AI-generated code
- Data privacy and confidentiality practices with third-party AI tools
- The same kind of model from Module 13A, now doing a different job

# AI-Assisted Coding Workflows

- Code generation: writing new code from a description — inventing structure from nothing
- Refactoring: restructuring code that already works — the risk is preserving behavior
- Test generation: writing a test for code just written — the risk is a test that proves nothing
- All three produce plausible, well-formatted output regardless of whether it's correct

**HANDS-ON** Generate and refactor a capstone feature's code with an AI assistant

**HANDS-ON** Generate a unit test for that feature with the assistant, then run it




*All three feed the same unverified “looks right” output*

PROBLEM

# Generated Code Looks Right. Is It?

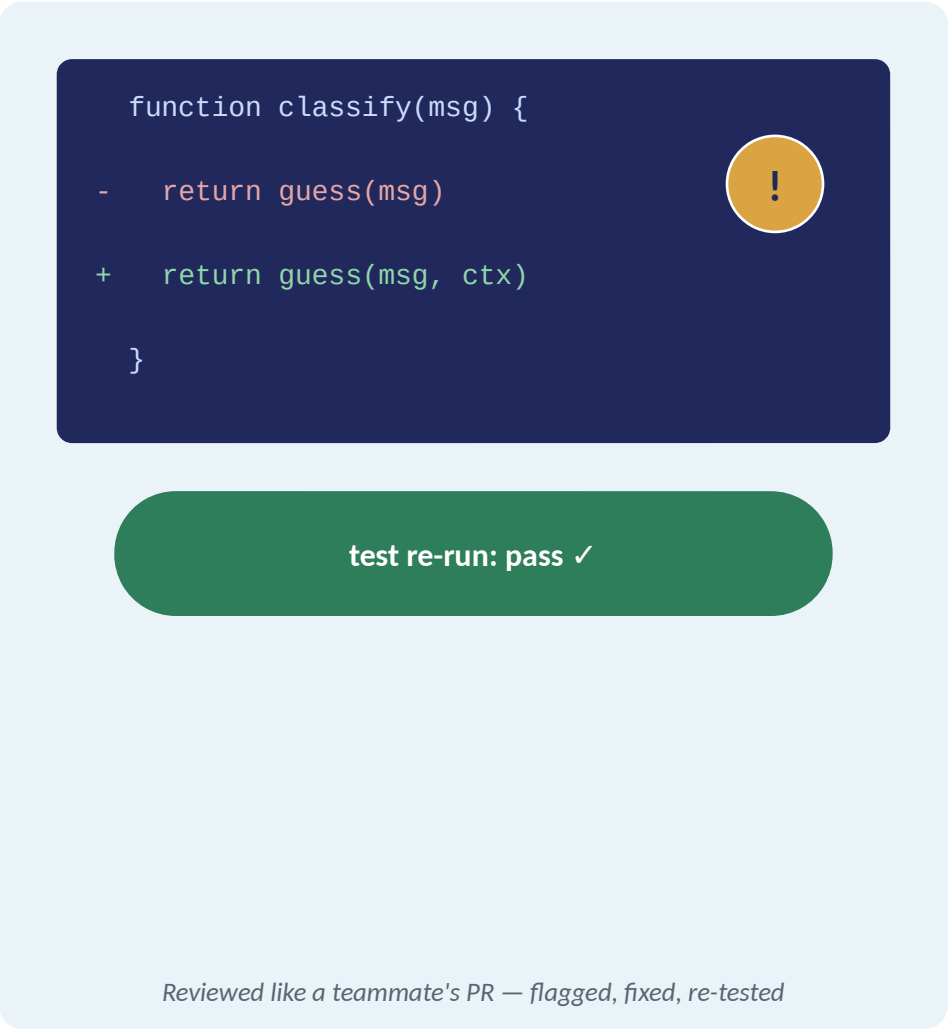
- A generated function, refactor, or test can look completely plausible
- “Compiles and looks clean” is not the same as “was reviewed”
- Merging without a real check repeats Module 13A's hallucination mistake — in code this time
- The team needs the same discipline it already uses for a human contributor's PR


*Passing checks alone doesn't mean reviewed*

SOLUTION

# Review Before You Merge

- Read a generated function line by line before accepting it — never just skim
- A refactor: run the existing tests before and after — did behavior actually survive?
- A generated test: break the code on purpose and confirm the test actually fails
- “Use it as a tool, not a replacement” for that review, every time

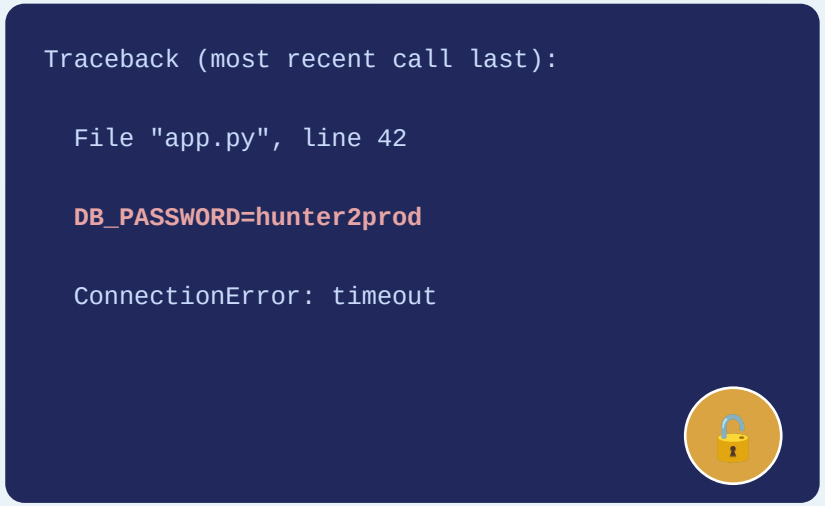



**HANDS-ON** Review the AI-generated diff and fix one issue before it merges

PROBLEM

# What Did You Just Paste Into That Chat Window?

- Debugging is fast when you paste a stack trace or config straight into the assistant's chat
- That's easy to do without thinking about what just left the building
- “Don't commit secrets to git” is a habit this audience already has
- “Don't paste secrets into an AI chat window” is the same rule — but it doesn't transfer on its own

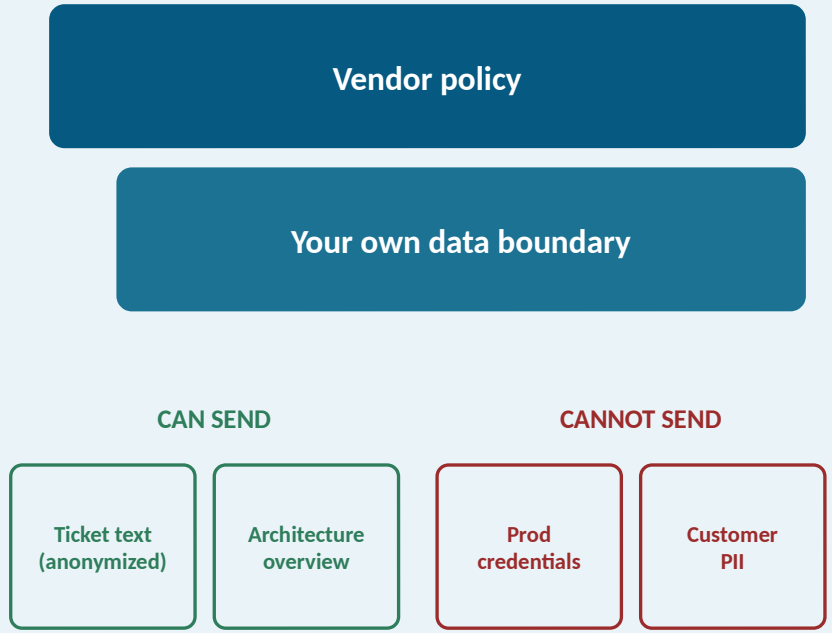



*A stack trace pasted straight into the chat window — secret and all*

SOLUTION

# Know Both Layers: Vendor Defaults & Your Own Boundary

- OpenAI: “by default, we do not train on any inputs or outputs… including… the API”
- Anthropic: “retained data is never used for model training without your express permission”
- A third-party model behind your assistant may carry a second, separate privacy policy
- “The vendor doesn't train on it” ≠ “it was fine to send” — that second call is the team's own




*Two layers of the same boundary — vendor default, then your own call*

**HANDS-ON** Write a data-handling note: what can and cannot go to a third-party AI tool

*Sources: GitHub Copilot Chat responsible-use guide; OpenAI & Anthropic data-retention policies.*